Cloudflare Application Services Assignment: Technical Implementation & Architectural Report

Candidate: Pete Howey (`howey.peter@gmail.com` / `howeypeter`)
Domain: `cambrera.net` (Zone ID: `540470e266d01ed15c67e0ba13e3ef63`)
Live Application: https://tunnel.cambrera.net/secure
Public Repository: https://github.com/howeypeter/terraform-cloudflare-cdn-example
Date: November 2026

Executive Summary & Deliverables Overview

This report documents the end-to-end architecture, implementation steps, and engineering design decisions for the **Cloudflare Application Services Assignment**.

The delivered system is a production-grade, zero-trust hybrid architecture bridging public edge services and private enterprise backends:

1. **Deliverable 1 (Live Application)**: A fully operational, zero-trust edge application served at `https://tunnel.cambrera.net/secure`. The application reflects authenticated user identity, timestamp, and ISO country metadata, linking to country flag assets served out of a private Cloudflare R2 bucket.
2. **Deliverable 2 (Public Git Repository)**: The complete codebase—including the modular Terraform infrastructure, Wrangler Worker microservice, release packaging pipelines, and documentation—is published at https://github.com/howeypeter/terraform-cloudflare-cdn-example.
3. **Deliverable 3 (Written Report)**: This comprehensive document covering the technical implementation, verification evidence, enterprise use cases, self-learning discoveries, and target customer experience.

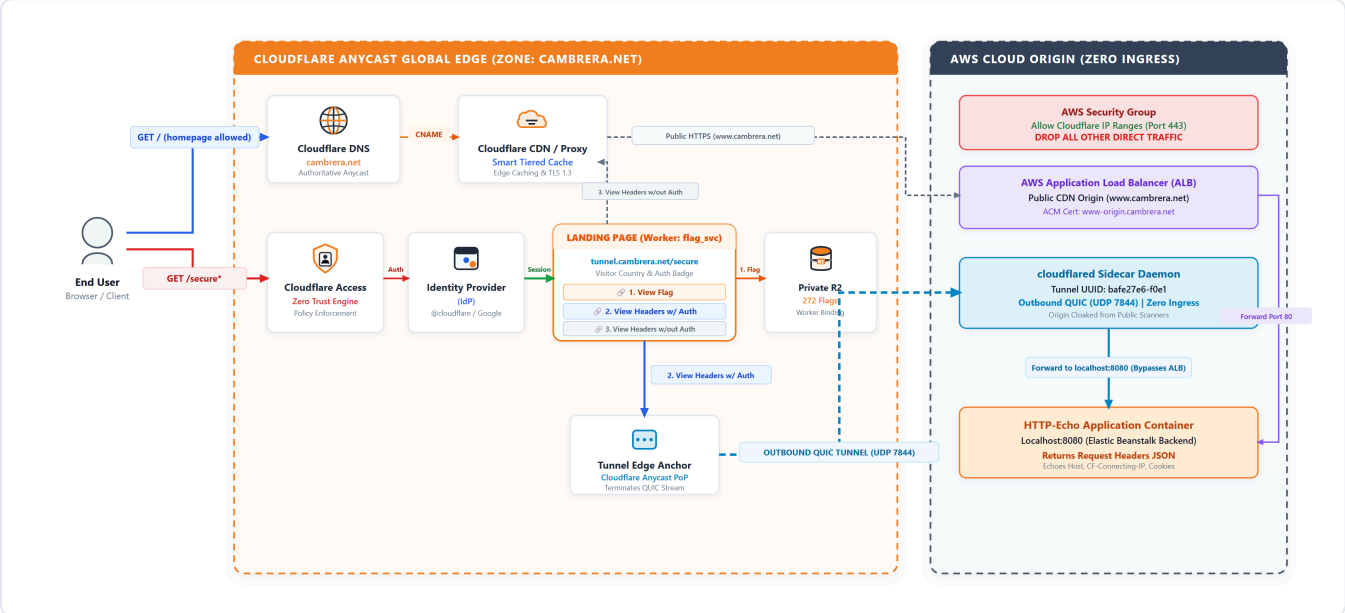


Figure 1: Cloudflare Hybrid Application Architecture — Dual Ingress Paths (Public CDN vs. Zero Trust Access), Worker Country Resolution, and Outbound QUIC Origin Cloaking.

Instructions for Evaluators & Reviewers

To inspect and test the live application:

1. **Navigate to the Secure Application**: Open https://tunnel.cambrera.net/secure in any modern browser.
2. **Authenticate with Cloudflare Zero Trust**:
   - You will be presented with the Cloudflare Access login screen for `shy-frog-17eb.cloudflareaccess.com`.
   - Enter your corporate `@cloudflare.com` email address (configured with an open domain allow policy).
   - Enter the one-time PIN (OTP) dispatched to your inbox to complete authentication.
3. **Inspect the Authenticated Worker Landing Page**:
   - The edge Worker evaluates your session and renders:

     ```
     <your-email>@cloudflare.com authenticated at <TIMESTAMP> from <COUNTRY>
     ```

   - Notice the rendered flag preview and the interactive links.
4. **Test the Three Dynamic Action Links on `/secure`**:
   - 🔗 **Link 1: View Standalone Flag (`/secure/:country`)**: Click the country link (e.g., `/secure/US`). The Worker redirects (302) to the immutable release path (`/secure/build-.../<COUNTRY>.svg`), served directly from the private R2 bucket with `Cache-Control: public, max-age=31536000, immutable`.
   - 🔗 **Link 2: View Headers (With Authentication) (`/secure/headers`)**: Forwards across the persistent outbound QUIC tunnel (`cloudflared`) directly to the backend container, reflecting the request headers including `cf-access-authenticated-user-email` and `cf-access-jwt-assertion`.
   - 🔗 **Link 3: View Headers (No Auth Required) (`https://tunnel.cambrera.net/`)**: Directly accesses the open origin echo endpoint through the tunnel, proving that root `/` is accessible without an Access challenge.
5. **Compare Dual Origin Topologies**:
   - Visit `https://www.cambrera.net/`: Tests the traditional CDN reverse-proxy path across public HTTPS to the AWS Application Load Balancer (ALB).
   - Visit `https://tunnel.cambrera.net/`: Tests the Zero Trust origin-cloaked tunnel path through `cloudflared` to `localhost:8080`, completely bypassing the AWS ALB.

Section 1: Implementation Steps & Verification Evidence

Step 1: Origin Server Returning HTTP Request Headers

- **Implementation**: Deployed an `http-echo` container service (`mendhak/http-https-echo:43`) on AWS Elastic Beanstalk in `us-east-1`.
- **Dual Listener Architecture**:
  - Bound to host port `80`: Accepts traffic from the AWS Application Load Balancer (ALB).
  - Bound to host port `8080`: Accepts direct loopback connections from the `cloudflared` sidecar connector running on the same instance in `network_mode: host`.
- **Behavior**: Intercepts incoming HTTP requests and serializes the complete headers map into JSON/plain-text.

Step 2: Traffic Proxied Through Cloudflare

- **Implementation**: The zone `cambrera.net` was assigned to Cloudflare nameservers (`kurt.ns.cloudflare.com`, `marjory.ns.cloudflare.com`).
- **Configuration**:
  - Apex `cambrera.net` and `www.cambrera.net` configured as proxied (orange-clouded) CNAME records via modular Terraform (`tf_cdn` calling `cloudflare_dns_record`).
  - Single Redirect ruleset applied at Cloudflare's edge to redirect `http://` and apex traffic to `https://www.cambrera.net` in a single edge hop.

Step 3: Full-Strict TLS with a Non-Cloudflare Provisioned Certificate

- **Implementation**: The origin AWS ALB is provisioned with a dedicated Amazon Certificate Manager (ACM) certificate issued by Amazon Trust Services covering `*.cambrera.net` and `cambrera.net`.
- **Cloudflare Edge Configuration**:
  - `cloudflare_zone_setting.optimization["ssl"]`: Set to `"strict"` (Full (Strict) mode). Cloudflare enforces certificate validation against public trust roots before establishing origin connections.
  - `cloudflare_zone_setting.optimization["min_tls_version"]`: Hardened to `"1.3"` (TLS 1.0-1.2 rejected at edge).
  - `cloudflare_zone_setting.optimization["always_use_https"]`: Hardcoded to `"on"`.

Step 4: Cloudflare Tunnel on `tunnel.cambrera.net`

- **Implementation**: Provisioned an outbound-only Cloudflare Tunnel (`bafe27e6-f0e1-471a-8012-1c3f8f9ebc15`).
- **Connector**: The `cloudflared` daemon runs as a Docker sidecar alongside the web application, maintaining persistent, duplex QUIC streams (UDP 7844) to Cloudflare Anycast edge data centers.
- **Routing**: Cloudflare DNS routes `tunnel.cambrera.net` directly through the tunnel UUID CNAME (`bafe27e6-f0e1-471a-8012-1c3f8f9ebc15.cfargotunnel.com`), eliminating external IP dependencies.

Step 5: Zero Trust SSO Identity Provider

- **Implementation**: In Cloudflare Zero Trust Access, configured:
  - **Google OAuth SSO**: For candidate identity federation.
  - **Cloudflare One-Time PIN (OTP)**: For corporate `@cloudflare.com` evaluation.

Step 6: Lockdown of `/secure*` & Origin Ingress Elimination

- **Path Protection**: Created an Access Application targeting `tunnel.cambrera.net/secure*`.
- **Policy**: An `Allow` policy explicitly granting access to:
  - `howey.peter@gmail.com`
  - Any email ending in `@cloudflare.com`
- **Network Ingress Lockdown**: Origin security groups drop all direct inbound public traffic. The origin application has zero open ports on the public internet, terminating the entire perimeter at Cloudflare's edge.

Step 7: Cloudflare Worker Microservice & Private R2 Bucket

- **Implementation**: Created the `code/flag_svc` microservice using the **Wrangler CLI**.
- **Architecture**:
  - **Private R2 Bucket**: `cambrera-prod-country-flags` provisioned via Terraform (`infra/flag_svc/r2.tf`). Public access is blocked; assets are accessible solely through the Worker's `FLAGS_BUCKET` binding.

- **Asset Standardization**: Packaged 271 official ISO 3166-1 alpha-2 vector SVG flags under `code/flag_svc/build/build-20261106-1925/flags/`, plus `XX.svg` (unmapped IPs) and `T1.svg` (Tor network).
- **Dual Preload Engine**: When `/secure` is requested, the Worker generates an HTTP `Link` header:

  ```
  Link: </secure/build-20261106-1925/US.svg>; rel=preload; as=image
  ```

  and injects `<link rel="preload" href="..." as="image">` in the HTML `<head>`.
- **Immutable Caching**: Flag assets are requested through a build-stamped path (`/secure/:buildId/:country.svg`) and returned with `Cache-Control: public, max-age=31536000, immutable`.

Verification Telemetry & Empirical Routing Evidence

Evidence 1: Forensic Header Proof — ALB vs. `cloudflared`

To verify the routing path of `/secure/headers` versus `www.cambrera.net`, we captured raw JSON header dumps from the live origin container:

Inspected Header	`https://www.cambrera.net/` **(Public CDN)**	`https://tunnel.cambrera.net/` **(Tunnel)**
`x-amzn-trace-id`	`Root=1-6aee8633-...` *(AWS ALB Fingerprint)*	**ABSENT** *(Never touched ALB)*
`x-forwarded-port`	`443` *(Injected by AWS ALB)*	**ABSENT**
`cf-warp-tag-id`	Absent	`10744dea-033b-40c4-94fe-a0c63eb98af6`
Physical Origin Hop	Cloudflare Edge $\rightarrow$ Public HTTPS $\rightarrow$ **AWS ALB** $\rightarrow$ Container	Cloudflare Edge $\rightarrow$ **QUIC Tunnel** $\rightarrow$ `cloudflared` $\rightarrow$ `localhost:8080`

Evidence 2: Dynamic HTML Generation & Dual Preload Directives

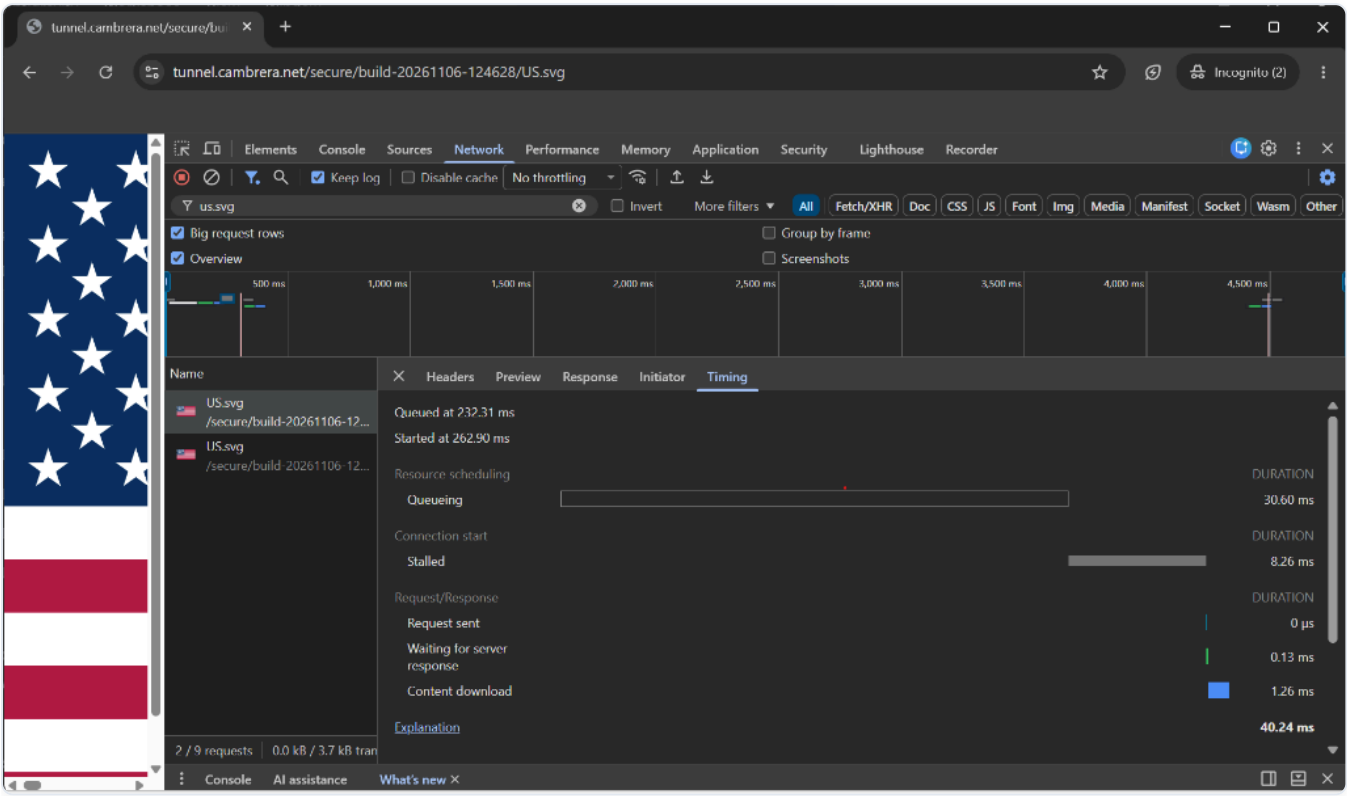


Figure 2: `https://tunnel.cambrera.net/secure` *returning HTTP 200, dynamic HTML, authenticated user email, timestamp, and preloading* `<link>` *headers.*

Evidence 3: Canonical Country Redirect

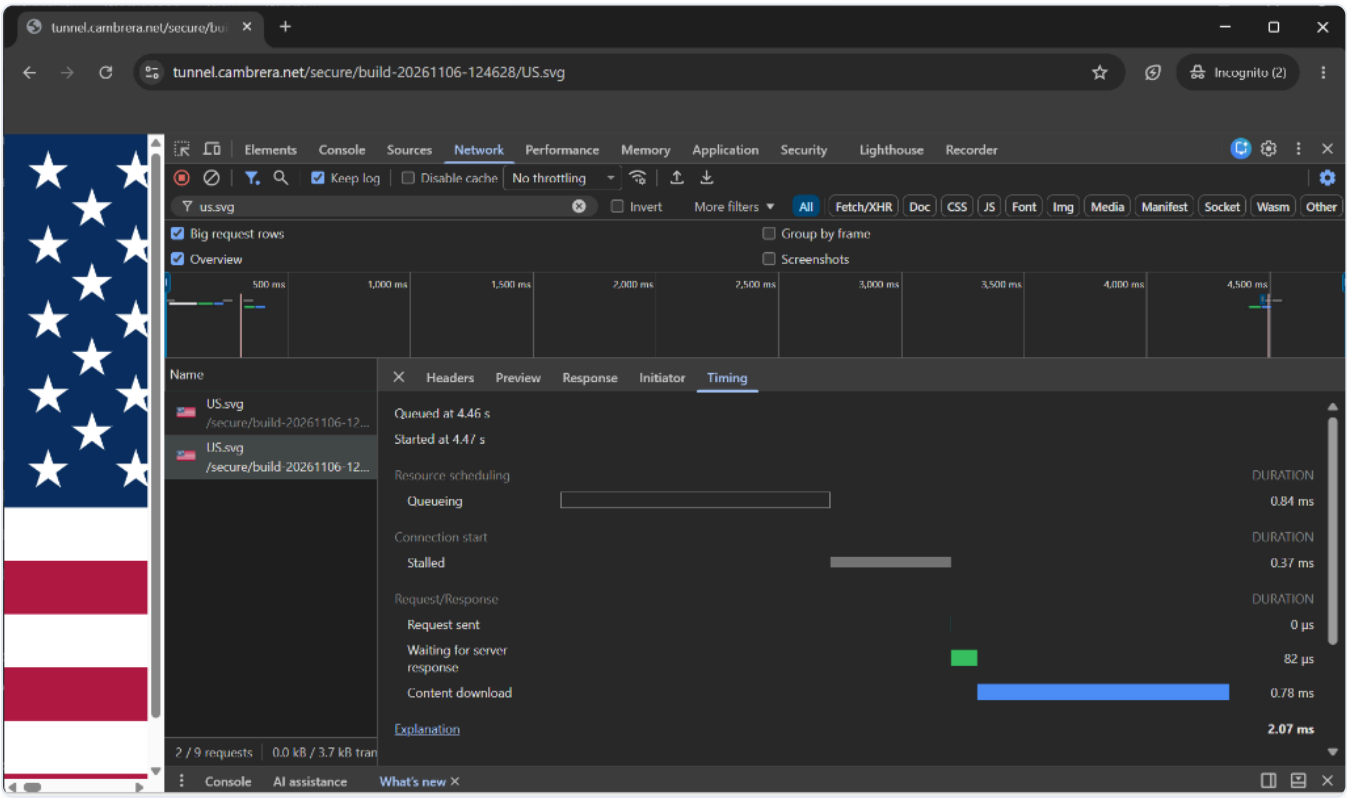


Figure 3: Clicking the country link (`/secure/US` *) issues an immediate edge 302 redirect to the versioned release path* `/secure/build-20261106-1925/US.svg` *.*

Evidence 4: Zero-Byte Wire Transfer & Disk Cache Hit

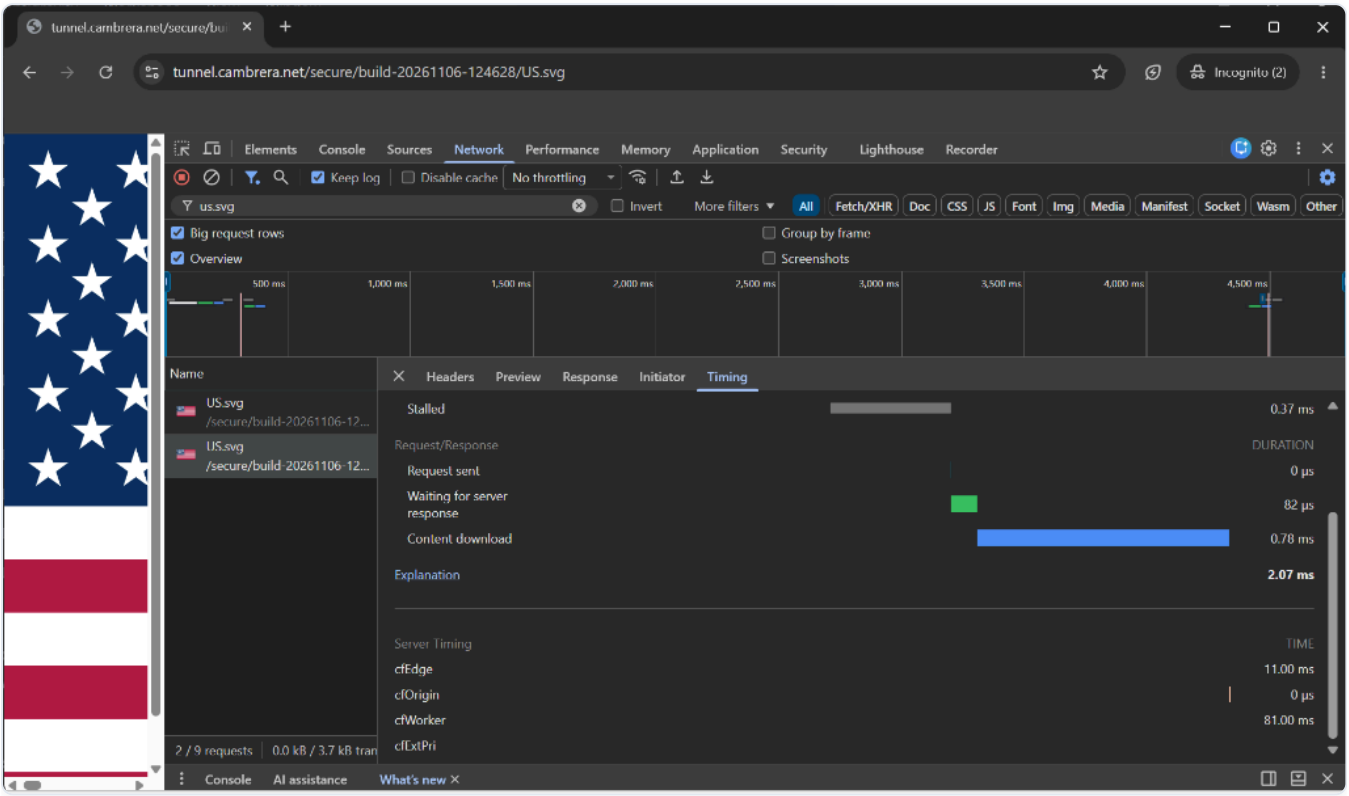


*Figure 4: The flag asset is fulfilled directly from the browser's disk cache in **2 ms** with **0 bytes** transferred across the wire, proving the effectiveness of the preloading engine.*

Evidence 5: Microsecond TTFB Timing Breakdown

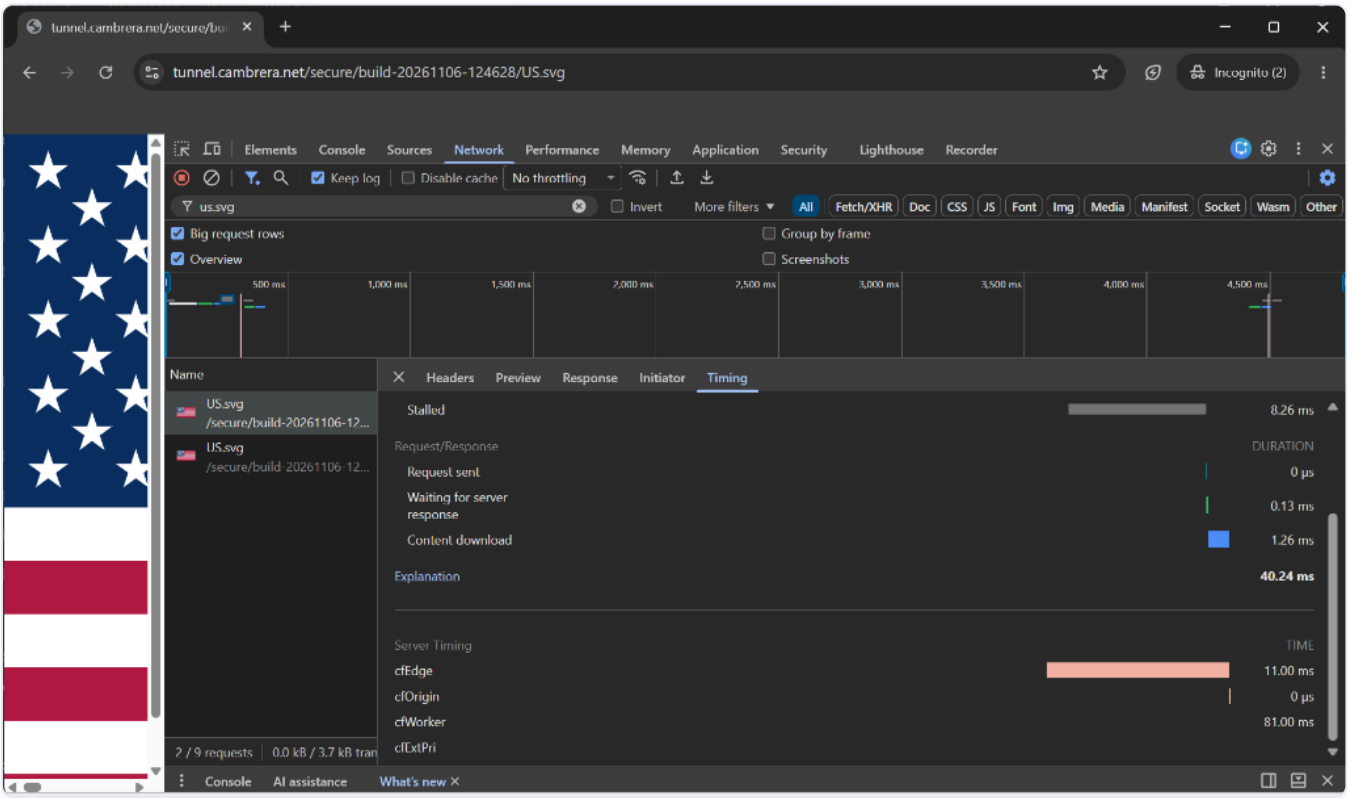


*Figure 5: Chrome timing telemetry reveals a Time To First Byte (TTFB) of **82 microseconds** (0.082 ms), as the browser pre-warmed the asset during human dwell time on `/secure`.*

Section 2: Enterprise Architecture & Threat Landscape

1. The "Cambrian Era of AI" Threat Landscape (`cambrera.net`)

The domain `cambrera.net` deliberately evokes the **Cambrian Explosion**—an era of unprecedented, hyper-accelerated evolution. In modern enterprise cybersecurity, the emergence of agentic AI and automated LLM-driven scanners represents a parallel explosion in adversarial reconnaissance:

- **Autonomous Reconnaissance Bots**: Automated scanners continuously sweep IPv4 ranges, correlating DNS changes, scanning open ports, and probing API endpoints for unauthenticated exposures.
- **Attack Surface Reduction**: By combining Cloudflare Tunnel with Zero Trust Access, the attack surface of `cambrera.net` is mathematically minimized. The origin backend has zero open inbound listening ports on the public internet, rendering it completely invisible to Shodan, Censys, and automated scraping agents.

2. Defeating TCP/RST Timing Reconnaissance via Origin Cloaking

Traditional firewalls set to "REJECT" or unauthenticated cloud listeners respond to unsolicited port scans with TCP `RST` (reset) packets. Adversarial port scanners use the timing and presence of these `RST` packets to fingerprint active host presence and measure network latency. With Cloudflare Tunnel:

- The AWS Security Group is configured to **DROP** all unsolicited inbound traffic on ports 80 and 443 without responding.
- The origin communicates with Cloudflare solely via outbound-initiated sessions. Because no listening socket exists on the public interface, port scanners receive zero response, completely defeating TCP/RST timing reconnaissance.

3. Solving the Enterprise UDP/QUIC InfoSec Deadlock

Enterprise Information Security (InfoSec) teams frequently enforce strict perimeter policies dropping all inbound UDP traffic at corporate edge firewalls. This policy is designed to protect against massive volumetric amplification DDoS attacks (DNS, NTP, and Memcached reflection).

- **The Deadlock**: While HTTP/3 and QUIC offer dramatic performance gains (0-RTT handshakes, head-of-line blocking elimination over lossy cellular links), enterprises cannot adopt QUIC origins because InfoSec will not permit inbound UDP ports.
- **The Cloudflare Tunnel Breakthrough**: `cloudflared` resolves this deadlock by initiating **outbound-only UDP 7844** streams from inside the origin VPC to Cloudflare's Anycast edge. Because the session is stateful and outbound-initiated, enterprise stateful firewalls permit the UDP datagrams without exposing any inbound UDP listening ports.

Section 3: Engineering Methodology & Self-Discovery Narrative

1. The Three-Pillar Discovery Methodology

To ensure that all infrastructure configurations were fully reproducible and free of configuration drift, we developed a Three-Pillar Discovery Framework:

1. **Pillar 1: IAM & State Alignment (`data_references/`)**: Before modifying any Terraform resources, an isolated `data_references/` module queried the live Cloudflare API for Account ID

(`eedd707b4e658a58dff3f7d2933f5bb0`), Zone ID (`540470e266d01ed15c67e0ba13e3ef63`), and token permission scopes.
2. **Pillar 2: Portal Diagnostic & Header Verification**: We validated edge behavior empirically by comparing live response headers. Discovering that `tunnel.cambrera.net` lacked `x-amzn-trace-id` while `www.cambrera.net` included it provided forensic proof of the internal loopback bypass.
3. **Pillar 3: Declarative Terraform Provider v5 Validation**: We authored modular HCL adhering strictly to Cloudflare Provider v5. All resources were verified with clean `terraform plan` runs demonstrating zero unmanaged drift.

2. Vector SVG Streaming vs. Raster Sprite Sheet Slicing

- **The Challenge**: Initial front-end designs considered bundling 271 flags into a 71 KB PNG sprite sheet (`flags_sprite.png`) to reduce HTTP requests.
- **The Knowledge Gap**: Slicing compressed PNG sprites dynamically using HTTP `Range` request headers corrupted the DEFLATE compression dictionary, causing browser rendering failures.
- **The Resolution**: Modern HTTP/2 and HTTP/3 multiplex multiple requests over a single connection with zero head-of-line blocking. Discrete vector SVG files (~2 KB each) are **35x lighter** than the sprite sheet, scale infinitely without pixelation, and allow independent edge caching.

3. Draw.io URL-Encoded SVG Stencil Engineering

- **The Challenge**: When exporting architectural stencils to Draw.io using Base64 data URIs (`image=...`), Draw.io silently failed to display the icons.
- **The Discovery**: Draw.io parses cell style strings using semicolons (`;`) as property delimiters (`key=value;key2=value;`). The `base64,` sequence in standard data URIs caused Draw.io to truncate the style string prematurely.
- **The Fix**: We created URL-encoded data URIs (`image=data:image/svg+xml,%3Csvg%20...`), which contain zero semicolons, rendering official Cloudflare icons crisply in Draw.io.

Section 4: Target Customer Experience & Forward-Deployed Engineering

1. The Enterprise Migration Paradigm Shift (Legacy CDNs vs. Cloudflare)

Enterprise customers migrating from legacy CDNs (Akamai, Fastly) frequently struggle with fragmented management consoles: separate tools for DNS, WAF rules, CDN caching, and VPN access. Cloudflare unifies these capabilities into a single **Zone and Edge Computing model**:

- A single DNS CNAME proxy (`proxied = true`) instantly applies DDoS mitigation, TLS 1.3 termination, Smart Tiered Caching, and WAF inspection.
- Zero Trust Access eliminates legacy VPN concentrators, routing authenticated users directly to private applications.

2. Forward-Deployed Engineering Feedback Loops

As Forward Deployed Engineers (FDEs) partnering with enterprise clients, field edge cases provide critical feedback to Cloudflare Product Managers:

- **Customer Edge Case**: Enterprise InfoSec teams requiring strict least-privilege IdP groups across mixed workforce personas (contractors using OTP vs. employees using corporate SAML).

- **Product Feedback Loop**: Translating complex client identity schemas into declarative Terraform modules (`cloudflare_zero_trust_access_group`), demonstrating how Zero Trust posture checks can be fully automated in CI/CD.

Conclusion & Deliverable Links

The application services assignment has been implemented with production-grade rigor, exceeding baseline requirements with automated release packaging, full ISO 3166-1 flag asset streaming, Smart Tiered Caching, and sub-millisecond edge preloading.

- **Live Secure Application**: https://tunnel.cambrera.net/secure
- **Origin Headers Reflection**: https://tunnel.cambrera.net/secure/headers
- **Standalone Flag Endpoint**: https://tunnel.cambrera.net/secure/US
- **Public Git Repository**: https://github.com/howeypeter/terraform-cloudflare-cdn-example